

Design Document: Multi-Armed Bandit Selection Mode for A/B Testing

Field	Value
Feature	Multi-Armed Bandit (MAB) Selection Mode for AB Testing
Version	Unica Interact 26.2
Date	June 2026
Status	Draft
Components	InteractDT (Design Time), RTP (RealTimePersonalization), Interact Runtime

1. Overview

A multi-armed bandit (MAB) is a dynamic testing method that continuously shifts traffic toward the best-performing variation. Unlike traditional A/B testing, it prioritizes **maximizing conversions** over strict statistical significance. It automatically reduces traffic to poor performers, saving revenue while the test is running.

MAB algorithms balance two goals simultaneously:

- **Exploration:** Randomly testing all variations to gather data and find the best one.
- **Exploitation:** Directing most of your traffic to the variation with the highest proven success rate.

1.1 Background

The existing A/B Testing feature in Unica Interact supports three selection modes:

Mode	Value	Description
UseDefault	0	When threshold reached, use the configured default branch

Mode	Value	Description
AutoSelect	1	System automatically selects the best-performing branch
UseBase	2	Use the base/control branch

These modes provide static or semi-static branch selection. To move towards AI-driven decisioning, we introduce a **Multi-Armed Bandit** approach that dynamically balances exploration (trying all branches) and exploitation (favouring the best performer).

1.2 Objective

Implement a basic Multi-Armed Bandit (Epsilon-Greedy variant) selection mode that:

- Explores all branches initially using configured percentages
- After a presentation threshold is reached, shifts to exploiting the best-performing branch while maintaining a configurable exploration rate
- Dynamically re-evaluates the best branch based on **recent** acceptance rate data

2. Functional Requirements

2.1 New Selection Mode

Add `MultiArmedBandit(3)` to the `ABTestSelectionMode` enum.

2.2 New Configuration Parameters (per AB Test)

Parameter	Type	Description	Range
Threshold	INTEGER	Total presentations across all branches before bandit logic activates	Default: ?
MinPercent	DOUBLE	Percentage of getOffers calls that continue random exploration after threshold is reached	0-100

Parameter	Type	Description	Range
Re-evaluation Interval	INTEGER	Interval in hours for re-evaluating the best performing branch	Minimum 1 hour.
Recent Transaction Weight	FLOAT	Weightage of recent transactions while calculating the acceptance rates	Decimal value between 0 and 1.

2.3 Behavior

□ Phase 1 (Exploration): `totalPresentations < threshold`
 → Select branch randomly based on configured sample percentages

Phase 2 (Exploit + Explore): `totalPresentations >= threshold`
 → With probability $(\text{minPercent} / 100)$: select randomly per configured percentages
 → With probability $(1 - (\text{minPercent} / 100))$: select the branch with highest recent acceptance rate

□ 2.4 Example

- 3 branches configured with percentages 40%, 35%, 25%
 - Threshold = 500, MinPercent = 10
 - For first 500 presentations:
 - random selection (40/35/25 distribution)
 - After 500 total presentations:
 - 10% of `getOffers` calls → random selection (40/35/25 distribution)
 - 90% of `getOffers` calls → best-performing branch selected
-

3. Design Time / RTP Changes

3.1 Database Schema Changes

3.1.1 ALTER TABLE — UACI_ABTest

Column	Type	Nullable	Default	Description
OtherSettings	Text	YES	NULL	JSON formatted settings for AB Test level. For MAB, we would store Threshold, MinPercent, Re-evaluation Interval, Recent Transaction Weight

ALTER TABLE UACI_ABTest ADD (OtherSettings VARCHAR2(1000) DEFAULT NULL);

Note: DDL scripts must be added/updated for all supported databases: Oracle, DB2, SQL Server, PostgreSQL, MariaDB, Informix.

3.2 UI Changes

The A/B Test configuration panel adds:

- **Selection Mode** dropdown: new option "Multi-Armed Bandit"
- When "Multi-Armed Bandit" is selected, show:
 - **Threshold** — numeric input (min: 1)
 - **Min Exploration %** — numeric input (range: 0–100)
 - **Re-evaluation Interval**
 - **Recent Transactions Weight**

4. Runtime Changes

Database Schema Changes

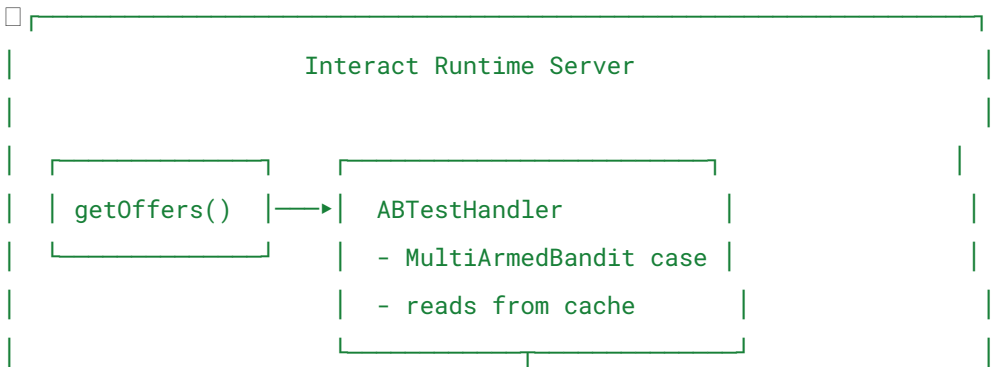
A new table need to be created in Interact runtime database:

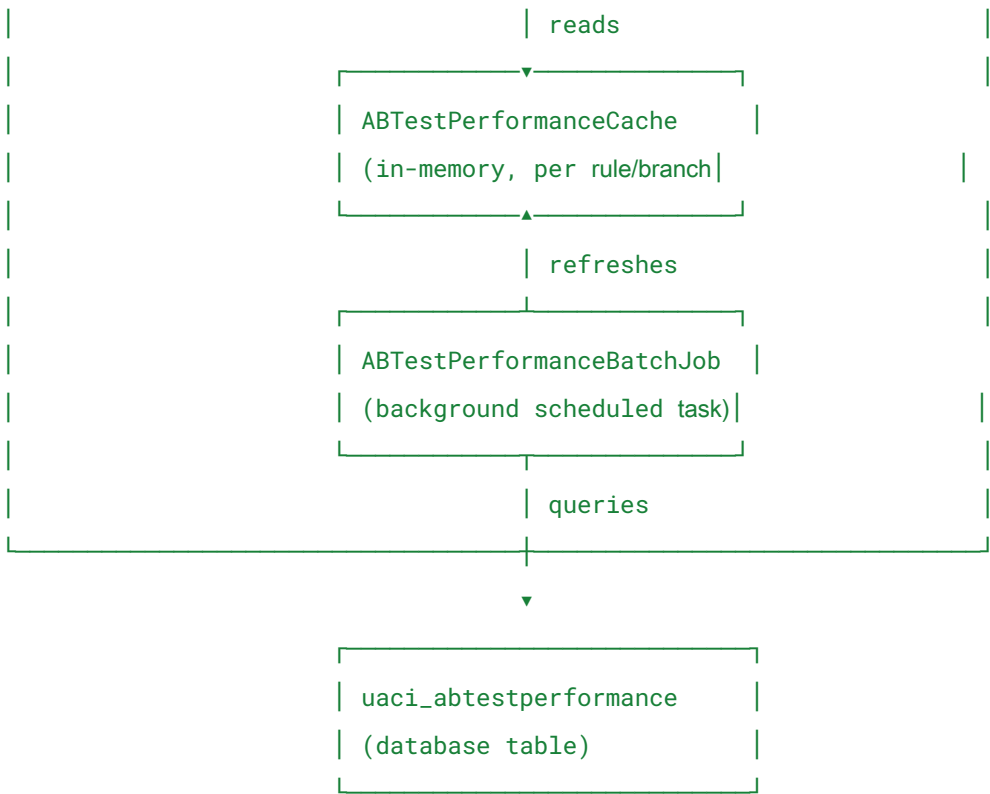
`uaci_abtestperformanceHistory` - need to be created to hold the last evaluation contactCount, acceptCount and UpdateDate. It would have all fields that are there in `uaci_abtestperformance` table:

Column	Type	Nullable	Default	Description
<code>RuleID</code>	Bigint	No		
<code>BranchID</code>	Bigint	No		
<code>ContactCount</code>	Bigint	No	0	
<code>AcceptCount</code>	Bigint	No	0	
<code>UpdateDate</code>	Datetime	No	Current Datetime	

A new column `acceptanceRate` need to be added to `uaci_abtestperformance` table to hold the calculated acceptance rates for each branch. All runtime instances simply read this table for abtest performance data.

4.1 Architecture Overview





4.2 Internal JVM Argument for testing (not to be documented)

Config Path	Type	Default	Description
<code>Com.hcl.interact.ABTestPerformanceBatchIntervalInHours</code>	FLOAT	1	Interval (hours) for the background job to refresh acceptance rate cache

4.3 Background Process — ABTestPerformanceBatchJob

Responsibilities:

- Runs on a scheduled interval (`ABTestPerformanceBatchIntervalInHours`)
- Use `uaci_tasklock` to make sure that only a single RT instance executes the batch.
- Queries `uaci_abtestperformance` for **overall** statistics and `uaci_abtestperformanceHistory` for last evaluation statistics.
- Computes acceptance rate per branch: `acceptanceRate = weight x (acceptance rate in the last interval) + (1.0 - weight) x (acceptance rate before the last interval)`

- 5. Update the `uaci_abtestperformance` and `uaci_abtestperformanceHistory` tables.

□4.4 Cache — `ABTestPerformanceCache`

We do not want to get the ABT performance stats during the `getOffers` call from database as it can impact performance of the API. We shall have a local cache for each runtime instance that shall read the database and acceptance rates for each branch in cache for `getOffers` to refer. The cache will be synced with the database based on the `ABTestPerformanceBatch` frequency.

5. Edge Cases and Error Handling

Scenario	Handling
All branches have 0 accepts	Fall back to random selection by percentage
Tied acceptance rates	Select the branch configured with higher percentage. If this is still a tie, select the first.
Best branch becomes disabled/expired	Re-evaluate; select next best from cache; if none available, fallback to random
Threshold = NULL for MAB mode	Validation prevents save;
MinPercent = 0	Valid — means 100% exploitation after threshold (no exploration)
MinPercent = 100	Valid — means always random selection (effectively no MAB)
Single branch remaining active	Always select that branch regardless of mode

6. Backward Compatibility

Concern	Mitigation
Existing AB tests (mode 0,1,2)	New columns are nullable with NULL default; no impact
Old runtime reading new DB	Ignores unknown columns; <code>getModeFromValue(3)</code> returns null → handler treats as no AB test (safe fallback)
Deployment data format	New fields are optional in deployment XML; old deployments remain valid
API backward compatibility	New fields are optional in REST/GraphQL; omitting them is valid for non-MAB modes

7. Testing Strategy

Key Test Scenarios

1. **Below threshold** — verify random distribution matches configured percentages
2. **At threshold boundary** — verify transition to exploit mode
3. **Epsilon exploration** — over many calls, verify ~minPercent% are random
4. **Best branch identification** — simulate different accept rates, verify correct winner
5. **Dynamic re-evaluation** — change acceptance pattern, verify cache update reflects new winner
6. **Lookback window** — verify old data is excluded from rate calculation

8. Risks and Mitigations

Risk	Impact	Mitigation
Cache stale after server restart	Suboptimal selections temporarily	Immediate batch run on startup; fallback to random

Risk	Impact	Mitigation
High-traffic load on background query	DB pressure	Query only MAB-configured rules; use indexed timestamp column
Epsilon too low in production	Insufficient exploration of new offers	Document recommendations; suggest minimum 5%
Threshold too low	Premature exploitation with insufficient data	Validate minimum threshold; recommend ≥ 100 per branch
